

Examen MVC Framework Development: API Setup

- Clone het MVC-project TandartsPraktijkAPI van Github Classroom

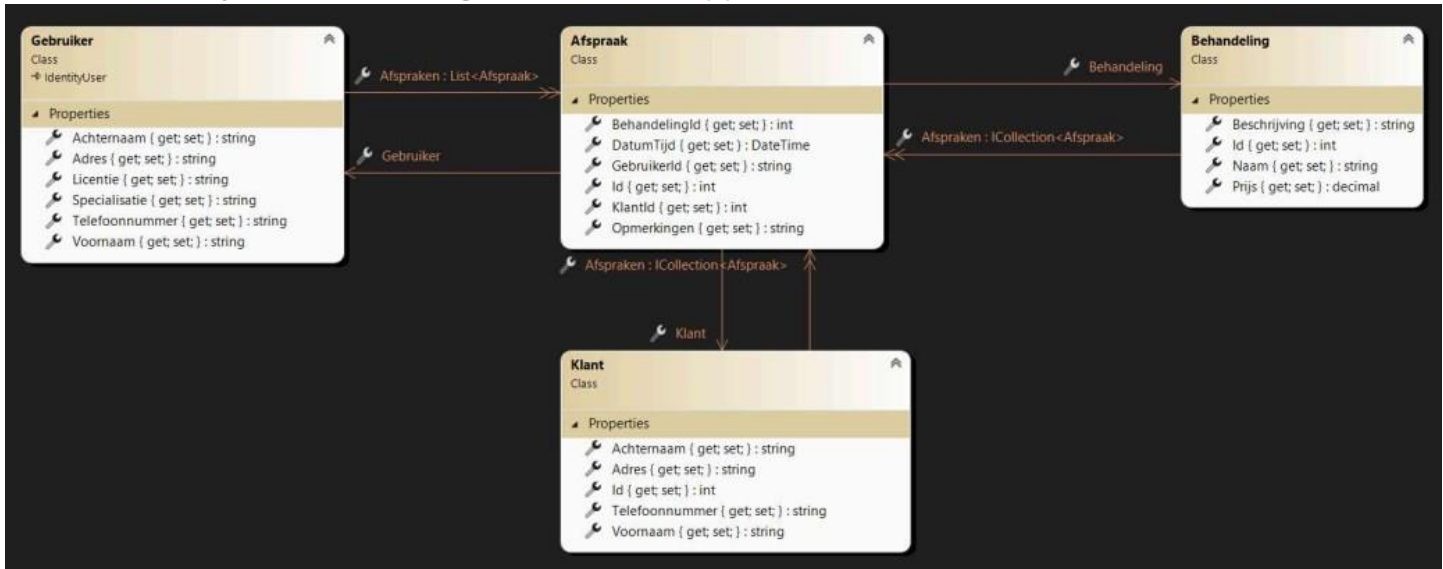
Je krijgt een applicatie voor een tandartspraktijk waar volgende elementen reeds aanwezig zijn:

- models
- databasecontext
- gebruikercontroller
- GlobalUsing.cs
- services in program.cs
- configuration voor automapper

De bedoeling is dat je deze applicatie vervolledigt zodat je afspraken en klanten kunt beheren. De applicatie moet beveiligd worden zodat enkel tandartsen en tandartsassistenten kunnen aanmelden.

1. Models

Hieronder vind je het klassendiagram voor deze applicatie.



Voeg de juiste Navigation properties toe volgens het klassendiagram.

Voeg volgende data-annotations toe en voeg **altijd gepaste foutmeldingen** toe:

model Klant

- Voornaam en Achternaam: verplicht, max 50 karakters
- Adres: verplicht, max 100 karakters
- Telefoonnummer: verplicht, max 15 karakters

model Gebruiker

- Voornaam, Achternaam en Licentie: verplicht, max 50 karakters
- Adres en Specialisatie: verplicht, max 100 karakters
- Telefoonnummer: verplicht, max 15 karakters

model Behandeling

- Naam: verplicht, max 100 karakters
- Prijs: verplicht, decimal (18,2), enkel van 0 tot 10000 mogelijk
- Beschrijving: verplicht, max 1000 karakters

model Afspraak

- DatumTijd: verplicht
- Opmerkingen: optioneel, max 500 karakters

2. DBContext + migraties

- Vervolledig de `APIDbContext`!
- Controleer je connectionstring in `AppSettings.Json`
- Maak een migratie *Initial create* aan.
- Update je database en voer het script `data\script\TandartsPraktijk.sql` uit om gegevens toe te voegen aan de database.

Dit script bevat ook reeds gebruikers en rollen (Tandarts en TandartsAssistent).

Als je wil aanmelden met een gebruiker, moet je **eerst het wachtwoord van de gebruiker wijzigen**. Je kunt hiervoor de methode `ResetUserPassword` gebruiken uit de gebruikerscontroller.

3. UOW en repositories

- Maak een specifieke repository aan voor de entiteiten `Klant` en `Afspraak` die overerven van de generieke repository.
- Voorzie een methode `Task ExistsAsync(int id)` in de generieke repository die een record opzoekt op basis van de meegegeven `id` en dan `true` of `false` teruggeeft als er een record gevonden wordt of niet.
- Vervolledig de `UnitOfWork` met repositories voor `Klant`, `Afspraak` en `Behandeling`. Voeg ook een methode `SaveChangesAsync` om gegevens weg te schrijven naar de database.

Je gebruikt in de controllers enkel methoden uit de `UnitOfWork`.

4. Controllers

Werk onderstaande controllers uit en gebruik, waar nodig, dto's en automapper. Gebruik validatie om te controleren of je gegevens kunt ophalen. Geef steeds correcte en gebruiksvriendelijke foutmeldingen.

AfspraakController

Één bepaalde afspraak ophalen

Maak een methode `GetAfspraak(int id)` die de gegevens ophaalt van één bepaalde afspraak en dan weergeeft volgens onderstaand voorbeeld.

GET `/api/Afspraak/{id}`

Parameters

Name	Description
id * required integer(\$int32) (path)	1

Execute

Responses

Curl

```
curl -X 'GET' \  
  'https://localhost:7241/api/Afspraak/1' \  
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:7241/api/Afspraak/1
```

Server response

Code	Details
200	Response body <pre>{ "tijdstip": "2024-10-20T09:30:00", "naamTandarts": "Jan Vermeer", "naamKlant": "Tom Pietersen", "behandeling": "Tanden Reinigen" }</pre>

Alle beschikbare tandartsen ophalen van een bepaald tijdstip

Maak een methode *GetBeschikbareGebruikersOpTijdstipAsync(DateTime tijdstip)* die de gebruikers toont die geen afspraak hebben ingepland vanaf een bepaald tijdstip tot een kwartier na het tijdstip. Controleer enkel de gebruikers die de rol *Tandarts* hebben. Maak hiervoor gebruik van de *userManager*.

Let op dat je in je in Swagger de datum moet ingeven volgens formaat mm-dd-yyyy. Achterliggend wordt dat verwerkt naar formaat dd-mm-yyyy zonder dat je daar iets voor hoeft te doen.

GET `/api/Afspraak/Beschikbare tandartsen`

Parameters

Name	Description
<code>tijdstip</code> <code>string(\$date-time)</code> <i>(query)</i>	10-24-2024

Execute

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7241/api/Afspraak/Beschikbare tandartsen?tijdstip=10-24-2024' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:7241/api/Afspraak/Beschikbare tandartsen?tijdstip=10-24-2024
```

Server response

Code	Details
200	Response body <pre>[{ "naam": "Jan Vermeer" }, { "naam": "Els Jansen" }]</pre>

Afspraak toevoegen

Maak een methode `CreateAfspraak(CreateAfspraakDto dto)` om een nieuwe gebruiker aan te maken volgens onderstaand voorbeeld. Valideer het model en controleer of het emailadres van de tandarts bestaat. Gebruik de methode `ExistsAsync` om te controleren of de behandeling en de klant bestaan.

POST

/api/Afspraak/Nieuwe afspraak

Parameters

No parameters

Request body

```
{  "datumTijd": "2024-10-18T12:15:55.468Z",  "opmerkingen": "/",  "tandartsEmail": "jan@example.com",  "klantId": 3,  "behandelingId": 3}
```

Afspraak wijzigen

Maak een methode `UpdateAfspraak(int id, UpdateAfspraakDto dto)` om een nieuwe gebruiker aan te maken volgens onderstaand voorbeeld. Valideer het model en controleer of het emailadres van de tandarts bestaat. Gebruik de methode `ExistsAsync` om te controleren of de behandeling en de klant bestaan.

PUT

/api/Afspraak/Afspraak wijzigen {id}

Parameters

Name	Description
id * required integer(\$int32) (path)	13

Request body

```
{  "id": 13,  "datumTijd": "2024-10-18T12:30:06.264Z",  "opmerkingen": "",  "tandartsEmail": "jan@example.com",  "klantId": 2,  "behandelingId": 2}
```

KlantController

Klanten met afspraken ophalen

Maak een methode *KlantenOphalenMetAfspraken()* om alle klanten met hun afspraken op te halen. Je geeft een overzicht zoals dit voorbeeld:

GET /api/Klant/Alle klanten met hun afspraken ophalen

Parameters

No parameters

Execute

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7241/api/Klant/Alle klanten met hun afspraken ophalen' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:7241/api/Klant/Alle klanten met hun afspraken ophalen
```

Server response

Code	Details
200	Response body <pre>[{ "naamKlant": "Lisa de Vries", "afspraken": [{ "datumAfspraak": "2024-10-21T11:00:00", "naamTandarts": "Jan Vermeer", "naamBehandeling": "Tanden Reinigen" }, { "datumAfspraak": "2024-10-18T12:30:06.264", "naamTandarts": "Jan Vermeer", "naamBehandeling": "Wortelkanaalbehandeling" }] }, { </pre>

Klant verwijderen

Maak een methode *DeleteKlant(int id)* om een klant te verwijderen. Controleer eerst of de klant bestaat. Voor de klant verwijderd wordt, moet je eerst de afspraken van die klant verwijderen.

5. Controllers beveiligen

- Beveilig de AfspraakController zodat elke methode enkel toegankelijk is voor een aangemelde gebruiker.
- In de klantController heeft de Tandarts toegang tot elke methode, maar de TandartsAssistente enkel op het ophalen van de klanten met hun afspraken.
- In de gebruikersController heeft de Tandarts rechten op alle methoden maar is de methode *Login* wel toegankelijk voor iedereen.